AI Gateway — Five-Month Executive Summary

Arvan Cloud AI Gateway · May–September 2026 · Executive Report

The project in one paragraph

Over the last five months we designed and stress-tested an AI gateway: a layer that sits in front of many language models and decides, for every request, **which model should answer, how much of the conversation to keep, and what can be served from memory** — always under explicit time and cost budgets. The work moved from a technology survey, through a full architecture design, to eleven benchmark and experimental studies whose numbers now decide what ships and what stays off. This report is the short version; the full detail, with every table and confidence interval, is in the accompanying technical report.

Page 1 — The verdicts

#	WHAT WAS JUDGED	VERDICT	HEADLINE NUMBER	WHY
1	**Anchor routing** (pick the model from the question text)	**TURN ON**	78.67% at $0.002751/ query	Matches the best GPU router's quality, 24× faster (0.64 ms vs 15.2 ms), needs no GPU, and beats the always-GPT-4 floor on quality AND cost.
2	**Reordering the prompt for the cache**	**TURN ON**	46–99% cacheable	Free (2.5 ms), changes no words, better answers in 5 of 12 request kinds, worse in none. The one unambiguous win.
3	**Cascades on top of a good router**	**KEEP OFF by default**	+1.1 pts for 2.4× tokens	Escalation-on-disagreement escalates 38% of correct answers. A cascade helps a weak router, not a strong one.
4	**Summarizing old chat**	**KEEP OFF**	−43.3 vs +10.1 pts	Destroys answers when the question refers back to the chat; helps only on topic changes the gateway cannot detect.
5	**Rewriting the question**	**KEEP OFF**	0 of 12 better	Even a strong rewriting model improved none of the 12 request kinds and made 2 worse, while adding a model call per request.
6	**Removing unimportant words**	**ONLY LONG DOCUMENTS**	−54% tokens, −4.3 pts	Works above 5,000 tokens only. Without that gate, scores fall 24–51 points everywhere.
7	**Semantic cache in Persian**	**MEANING LAYER OFF**	63% wrong	The meaning-match layer served a wrong answer to nearly two of every three Persian reuses. Exact and keyword layers stay.
8	**The cache guard**	**PER LANGUAGE**	0.88:1 vs 6.5:1	A fair trade in Persian (0.88 good answers lost per bad one stopped), a bad one in English (6.5 lost per bad stopped).

How to read the numbers. Every quality score is out of 100 and every difference carries a confidence interval. Asking the same model the same question twice already moves scores by up to ±6 points, so we only call a change real when it clearly exceeds that noise. Where a number could not be established, we say so rather than guess.

Page 2 — What we built the decisions on

Scale of the evidence.

STUDY	WHAT IT TESTED	SIZE
RouterBench, nine routing methods	11 models, real dollar costs	36,481 prompts (7,296 held out)
ArvanCloud pool, routing + cascades	7 models, real list prices	150 MMLU-Pro questions (90 held out), 12 splits
PCM scenario test, four methods × 12 request kinds	Answer quality with each method on/off	3,464 requests per method
PCM ablation (Aug)	Compression vs routing accuracy	~1,600 judge calls, 100 prompts per arm
PCM ablation (Sep)	All four methods at 1,000 questions	~12,700 judge calls
Cache paraphrase benchmark	Five-layer cache correctness	9,917 labelled question pairs, 2 languages
Live cache + guard trial	Cache and guard on real traffic	5,000 prompts through the gateway

Page 3 — The five findings that matter most

1. A $0.0005 router question beats a $0.003 model call. The old baseline used an LLM judge to pick the model: 61.25% quality at $0.000223 per query and ~2,180 ms per decision. Our anchor router reaches 64.75% on the same prompts at $0.000143 — better quality, 36% cheaper, and about 0.4 ms instead of 2 seconds (sources 07 and 08).

2. Half the tokens, same routing accuracy — but only on long prompts. On ~6,000-token prompts our shipped compression cut judge input tokens 47.6% with exactly 0.00 points of routing-accuracy change. On the 178-token prompts that dominate real traffic it bought nothing and cost 41 ms, which is why the 5,000-token gate exists and must stay (sources 11 and 12).

3. We caught our own mistake before it shipped. An earlier round recommended chat summarization as "free" (+0.9 points). Re-tested on conversations where the question actually depends on history, it costs −40.8 points and destroyed 449 of 996 answers. The earlier test had been built from unrelated questions. This reversal — and the honesty of recording it — is the strongest evidence the evaluation process works (source 12).

4. Persian breaks the default tools, and we measured exactly how. Blind reuse would score 67% in English and 40% in Persian just from the data mix; after correcting for that, the cache adds +20.4 points in English but only +8.2 in Persian. The meaning-match layer was quietly using an English-only embedding model — now configurable, multilingual by default, and pinned by 129 automated tests (source 14).

5. The architecture makes budget violations structurally impossible. Every request runs on a ledger that reserves cost before dispatch and a watchdog that fires at deadline minus a reserved tail, cancelling in-flight work and returning the best answer so far. The design keeps the hot path boring — rules, hash lookups, ledger arithmetic, at most one 1.5B router model (source 06).

Page 4 — What to do next

1. **Ship anchor routing** as the default router, with rules as the permanent fallback floor. Decision time 0.64 ms on CPU; budget p99 ≤ 25 ms is met with ~40× headroom.

2. **Turn on prompt reordering everywhere.** Free money: cache share 0% → 46–99% depending on request kind, and the gateway is 40% faster under load (4.49 s → 2.68 s at 8 concurrent requests).
3. **Keep cascades available but not default.** anchor top-3 + cascade reaches 94.4% accuracy (vs 93.3% anchor alone, 92.7% best single model) but at 2.07 calls and 1,505 tokens per query; on 90 questions the interval spans zero. Use only where accuracy per question outweighs cost.
4. **Keep the three "off" verdicts off**: chat summarization, question rewriting, and sub-5,000-token word removal. Each failed its scenario test; two of them can silently corrupt answers.
5. **Set cache policy per language.** Persian: exact + keyword only, guard on. English: dial at 0.85 (88% right, 17% caught) or 0.95 (95% right, 4% caught), guard off.
6. **Close the two open measurement gaps before quoting savings externally**: re-run compression on real retrieved documents (current numbers are best cases on synthetic data), and re-baseline the semantic-cache hit-rate gate on real traffic.

Limits of this summary. One answering model (GPT-5.4-Mini) in the scenario tests; benchmark questions are already well-written; ~300 questions per request kind means changes under ~3 points are invisible; long-document and chat test sets are synthetic; ArvanCloud figures carry ~±6 pp variance on 90 held-out questions; OpenRouter list prices stood in for ArvanCloud pricing.

Page 5 — Source documents

Every document referenced above is copied, renamed, inside this folder:

REF	FILE	WHAT IT IS
01	`sources/01-core-technology-stack.pdf`	Technology stack selection (Python, FastAPI, LiteLLM, Redis, PostgreSQL, vLLM)
02	`sources/02-evaluation-metrics-reference.pdf`	Definitions of Win Rate, AUCQ, TTFT, TPOT, Pareto frontier
03	`sources/03-advanced-routing-methodologies-survey.pdf`	Survey of 11 advanced routing methodologies
04	`sources/04-routing-baselines.pdf`	Naive, heuristic, utility-based, and cascade baselines
05	`sources/05-verified-technique-inventory.pdf`	55 techniques across 7 areas, adversarially fact-checked
06	`sources/06-architecture-and-technique-analysis.pdf`	3-phase architecture, feature-flag ladder, assumption register
07	`sources/07-routerbench-three-routers-report.pdf`	First RouterBench study: 3 routers implemented and evaluated
08	`sources/08-routerbench-nine-methods-report.pdf`	Full nine-method RouterBench comparison
09	`sources/09-routing-cascading-synthesis.pdf`	Cross-study synthesis of 5 routing/cascade studies
10	`sources/10-arvancloud-routing-cascade-report.pdf`	Routing + cascade test on the 7-model ArvanCloud pool
11	`sources/11-pcm-ablation-august.pdf`	PCM ablation: compression vs routing accuracy (August)
12	`sources/12-pcm-ablation-september.pdf`	PCM ablation: four methods at 1,000 questions (September)
13	`sources/13-pcm-scenario-results.pdf`	PCM scenario test: 4 methods × 12 request kinds

REF	FILE	WHAT IT IS
14	`sources/14-cache-paraphrase-benchmark.pdf`	Five-layer cache benchmark, 9,917 pairs, 2 languages
15	`sources/15-cascade-vs-mini-only-comparison.md`	Nano→mini cascade vs mini-only, 20 questions
16	`sources/16-live-cache-guard-results.json`	Live 5,000-prompt gateway trial of cache + guard
17	`sources/17-summarization-and-speculative-decoding-survey/`	LaTeX survey: summarization & speculative decoding